

MINIMALISTIC AUTOSAR-INSPIRED LAYERED FIRMWARE ARCHITECTURE FOR DISTRIBUTED SYSTEMS

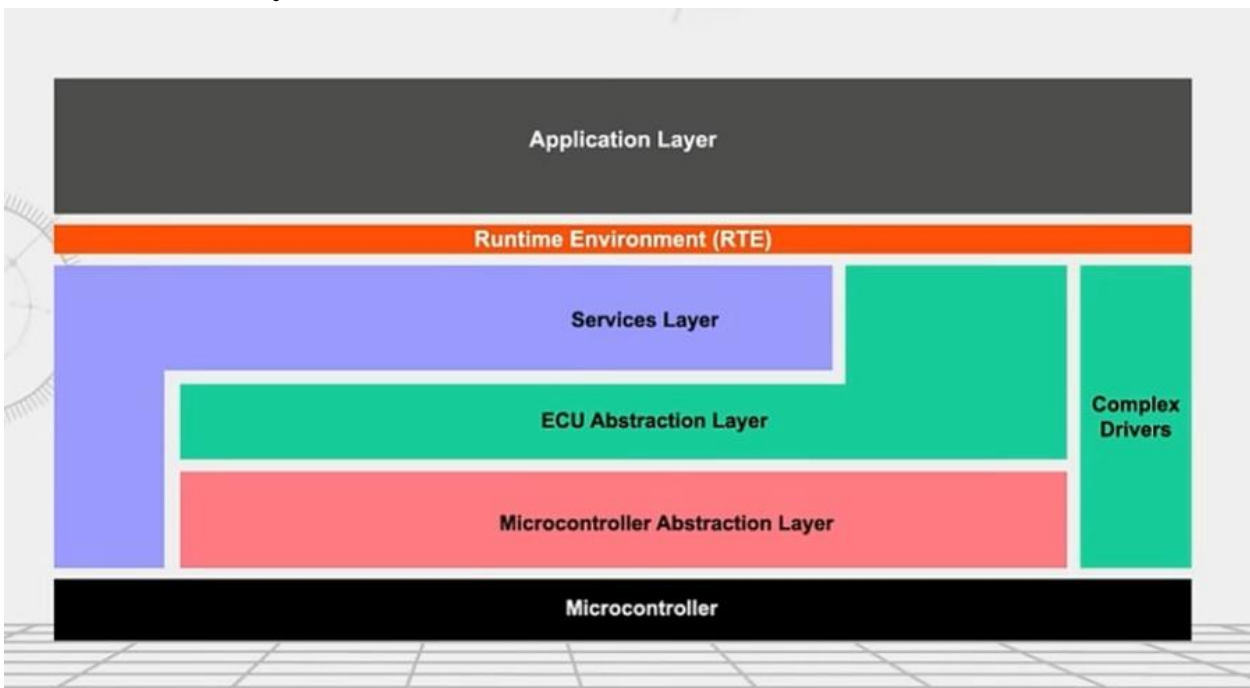
Problem Statement

Embedded applications developed for microcontrollers are frequently designed as a monolithic architecture, where application logic, peripheral drivers, communication mechanisms, and safety mechanisms are tightly intertwined. Such designs become increasingly difficult to scale, maintain, and reuse when multiple distributed controllers are required to cooperate. Although industrial frameworks such as AUTOSAR address these challenges through standardized software architecture, their complexity and licensing requirements make them impractical for academic learning and experimentation.

Proposed Solution

This project proposes the design and implementation of a lightweight, AUTOSAR-inspired firmware framework using FreeRTOS on STM32 microcontrollers. By recreating the fundamental concepts of AUTOSAR such as layered architecture, software component abstraction, driver initialization, runtime communication, and reusable basic services, the project demonstrates how modular embedded firmware can be developed for distributed real-time systems while remaining simple enough for educational purposes.

AUTOSAR Layered Architecture



AUTOSAR Framework

The framework follows the AUTOSAR philosophy by separating the application logic from hardware-specific implementation.

- **Configuration Layer:**
Stores system configuration, ECU mapping, task priorities, communication channels, and initialization parameters.
- **Software Components (SWCs):**
Modular application functions such as Dashboard Manager, Brake Control, Throttle Control, and Airbag Controller.
- **Runtime Environment (RTE):**
Provides standardized APIs for communication between SWCs and between ECUs.
- **Basic Software (BSW):**
Contains communication services, diagnostic services, memory services, watchdog, and OS abstraction.
- **AUTOSAR OS:**
Schedules tasks, timers, queues, semaphores, and inter-task communication.
- **MCU Drivers (MCAL):**
CAN, Ethernet, etc

ECU Architecture

- Configuration
- SWCs
- RTE
- BSW
- FreeRTOS
- MCU Drivers

ECU A – Dashboard ECU

ECU B – Throttle & Brake ECU

ECU C – Airbag ECU

Proposed Methodology

